CANDY
DIALOGUE CREATOR

Create & Edit Commands

1.0.0

Creating or editing commands is a streamlined and fairly easy process.

Note that it can only be done in the raw project form of Candy Dialogue Creator (i.e. in the uncompiled files). The raw project can be run directly from the Godot editor, but for performance reasons, you'll probably want to compile a new version of Candy DC with your added commands.

In the future, we'll implement a method to add custom commands without recompiling, and in an even simpler manner.

Command Data

The first step is to add the base information about the command you want to create.

Open **User_Variables.gd** and scroll down to the **line_colors** dictionary. This is where you'll define the colors to use for your command in the GUI.

Add your command under one of the existing categories (make sure the command's name starts with '§' as a prefix), and provide two colors: "BG" and "Border".

By default, commands use two color patterns: a colored background with a black outline, or a dark background with a colored outline. You're free to create any other pattern you like, just be aware of how it will display in the GUI: the default patterns and color schemes were chosen to improve readability.

Next, scroll further down to the **command_templates** dictionary. Add your command there, and once again make sure its name has the '§' symbol as a prefix. Only 'Spoken Line' never has that prefix.

In case it's not obvious, the goal here is to provide all the pieces of data (in the form of dictionary keys) that your command will need in Candy Dialogue Engine.

A few commands may have data that is only used in the Candy Dialogue Creator. Some commands are also missing data keys in the **command_templates** dictionary, as it is added elsewhere as needed (for example, for the §Choice_List command, Categories, Choices and Timers are added through the Choice Editor UI).

Data Fields

Next, you need to configure how the command line will be displayed in the GUI. Specifically: which data fields does it require.

Command Display Code

Open **Universal_Line.gd** and near the top, look at the **_apply_line_data_to_ui()** function.

The function begins with a long list of nodes (data fields) that are set to visible = false. If you create new data fields, you should add them there. The purpose of making these nodes invisible is so they don't appear inside commands that don't need them, as this would create confusion. Hence, by default, all data fields have their visibility disabled.

Scroll further down, until you reach the part of the function titled '#@ Display Universal Line'. This is where you'll determine how your command displays.

If you look at the default commands, they follow a similar template:

```
"§Call":
    $"HBox/Move".text = "Call"
    $"HBox/Move".tooltip_text += "Call a function."

    #% Get data:
    $"HBox/Con/Function".text = str(line_data.get("Function", ""))
    $"HBox/Con/Arguments".text = str(line_data.get("Arguments", ""))

    if line_data["Await"] == true:
        $"HBox/Con/Await".modulate = candy_dc.color_models[candy_dc.color_mode]["On"]   #/ green
    elif line_data["Await"] == false:
        $"HBox/Con/Await".modulate = candy_dc.color_models[candy_dc.color_mode]["Off"]   #/ red

    #% Toggle Visibility:
    $"HBox/Con".visible = true
    $"HBox/Con/Await".visible = true
    $"HBox/Con/Function".visible = true
    $"HBox/Con/Arguments".visible = true

    stylebox.bg_color = candy_dc.line_colors[key_name]["BG"]
    stylebox.border_color = candy_dc.line_colors[key_name]["Border"]
```

$"HBox/Move".text should be set to the name you want to display for the command, inside the 'Move' button. The 'Move' button is the button that each command line features, which displays the name of the command, and which you can click to select a command line, or right-click to cut/copy/paste a command line.

The name displayed in the 'Move' button can be anything you want: it doesn't need to match the actual command name exactly, and it doesn't need the '§' prefix.

You can also change the tooltip text of the 'Move' button if you want to, to provide a reminder as to what the command does.

The '#% Get data' section features code to fetch data from the command itself, and update the command line's data fields to display it correctly. There are generally just three kinds of data fields: LineEdit, Button, and OptionButton.

For LineEdit nodes, the code is straightforward. For example:

```
$"HBox/Con/Function".text = str(line_data.get("Function", ""))
```

The example above displays the value of the command's "Function" data key inside the Function LineEdit: "HBox/Con/Function" is the LineEdit node, and "Function" is the corresponding data keys of the command, from which the data is fetched.

For Button nodes, the code is still simple:

```
if line_data["Await"] == true:
      $"HBox/Con/Await".modulate = cdc.color_models[cdc.color_mode]["On"]
elif line_data["Await"] == false:
      $"HBox/Con/Await".modulate = cdc.color_models[cdc.color_mode]["Off"]
```

Basically, you use an 'if' condition to check the value of the corresponding data key, and based on this, you modify the color and/or text of the button.

We recommend using colors from candy_dc.color_models, for compatibility with color-blind mode.

For OptionButton nodes, just use the template below, replacing the data field node and the data key name as appropriate:

```
#% Format + Method:
_apply_option_value($"HBox/Con/Format", line_data, "Format")
_apply_option_value($"HBox/Con/Method", line_data, "Method")
```

```
_apply_option_value($"HBox/Con/Format", line_data, "Format")
```

You'll then need to indicate which data fields the command must display, under '#% Toggle visibility':

```
$"HBox/Con".visible = true
$"HBox/Con/Await".visible = true
$"HBox/Con/Function".visible = true
$"HBox/Con/Arguments".visible = true
```

Just replace the nodes in this example with the ones you want to display. Remember that data field nodes are organized into categories (e.g. $"HBox/Con"), so you must make those categories visible as well.

Finally, indicate which colors the command line should use, from the earlier **line_colors** dictionary:

```
stylebox.bg_color = candy_dc.line_colors[key_name]["BG"]
stylebox.border_color = candy_dc.line_colors[key_name]["Border"]
```

You'll need to specify one color for **stylebox.bg_color** (the line's background), and another for **stylebox.border_color** (the line's border).

In this case, no alternatives are built-in for color-blind mode because the options aren't limited to a small number of select colors. Thankfully, unlike button colors, line colors only provide convenience and don't convey important information.

Custom Data Fields

If your command requires creating new data fields, you'll need to do so in the Universal Line.tscn scene.

You should then link signals from your custom data fields to the **Universal_Line.gd** script. If your data fields consist of LineEdit, Button or OptionButton nodes, you can look at the existing functions for templates.

Note that if you want to add a (⁝) button to insert values from a drop-down list inside a LineEdit node, the procedure is a bit more involved. We won't get into this here but you can look at the code of such default data fields as an example. We might update this part of the code considerably at a later time, as we don't find the current implementation intuitive, so we'll provide proper instructions at that point.

Writer Script

You've now done the hardest part. The last remaining step to adding custom commands is editing the script of the Writer UI.

Open **Writer.gd** and near the top, look at the **show_hide_lines** dictionary.

```
var show_hide_lines = {
    "Comments": {
        "Hide": false,
        "Commands": ["§Comment"],
        "Color": Color(0.75, 0.75, 0.75)
    },
    "Choices": {
        "Hide": false,
        "Commands": ["§Choice_List", "§Choice_Status", "§Timer_Status"],
        "Color": Color(0.505, 0.382, 0.75)
    },
```

This dictionary is used for showing or hiding specific categories in the line overview list, to the left of the Writer UI. You may notice each top-level key matches a filtering button, it doesn't strictly match the command categories (for example "Comments" is listed as a separate category here, even though the "§Comment" command is normally part of the "Control Commands" category).

Simply add your custom command to whichever category whose button should hide or show it, inside the corresponding "Commands" array.

Finally, scroll down to the **update_spoken_lines_list()** function. Further down inside that function, you'll find a section titled '#@ Command Lines', with a 'match command_name:' statement.

This match statement determines what data is displayed, for each command, inside the line list on the left of the Writer UI:

```
#% Add extra info for certain commands:
match command_name:
    #?-----------------------------------------------------
    "§Comment":
        var comment_text := str(command_data["Comment"]).strip_edges()
        display_text = "%d | %s: %s" % [i, command_name, comment_text]

    #?-----------------------------------------------------
    "§Bridge", "§Jump":
        var convo_ref := str(command_data.get("Conversation", "")).strip_edges()
        var block_ref := str(command_data.get("Block", "")).strip_edges()
        var line_ref := str(command_data.get("Line", "")).strip_edges()
        display_text = "%d | %s: %s → %s → %s" % [i, command_name, convo_ref, block_ref, line_ref]

    #?-----------------------------------------------------
    "§LM":
        var ref_text := str(command_data["Reference"]).strip_edges()
        display_text = "%d | %s: %s" % [i, command_name, ref_text]
```

Add a match case for your command (don't forget the '§' prefix), and using the existing cases as templates, write what data should be displayed.

Not all command data needs to be displayed: the list of lines is an overview, meant for easy navigation while writing Spoken Lines, so we recommend limiting it to data that is actually most relevant.

You can also decide how the data is formatted. For example, you can add symbols such as ':' or '→', and display some data inside brackets, quotes or parentheses.

On a last note, if you scroll a little bit further down, you'll find this block:

```
#% Adjust colors:
if font_col == Color(0.2, 0.2, 0.2) or font_col == Color(0.25, 0.25, 0.25):
      font_col = Color(0, 0, 0)
```

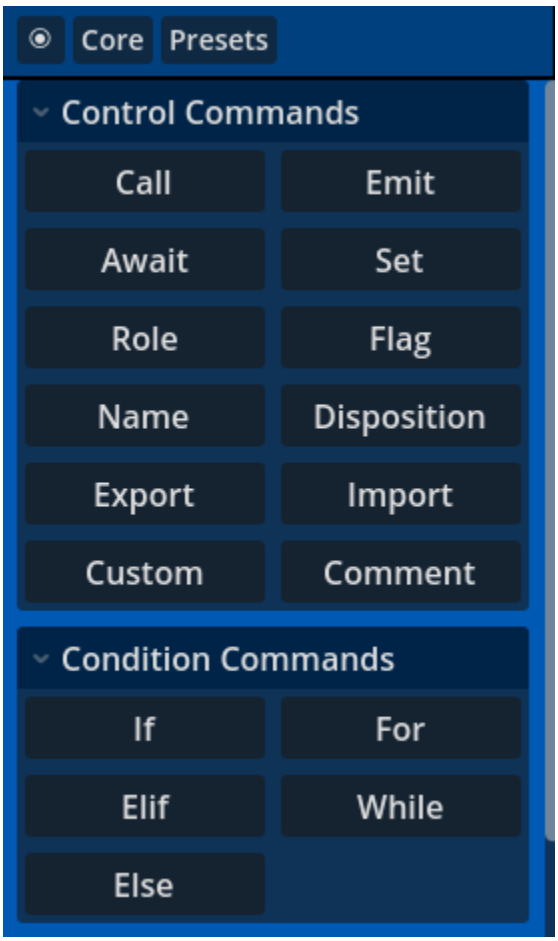
This makes the font show in full black, for commands that have a dark gray background matching either Color(0.2, 0.2, 0.2) or Color(0.25, 0.25, 0.25).

You can add your own background colors to the 'if' condition, for which you'd like the font to appear full black, but be aware that it applies to all commands.

UI Button

The last step is to add a button in the UI, so you can add your command.

Open the **Main UI.tscn** scene, and look at the left margin where all the command buttons are:



Follow these steps:

1. Duplicate any button in the category that matches your command, and rename the button node to the name of your command, without the '§' prefix.
2. Change the text of the button to match your command (purely descriptive: it doesn't need to be an exact match to the command's name).
3. Reorder the button if you want to.
4. Don't connect the button's signals. The code will do it automatically at startup.

Finish

That's it. If you've done everything correctly, it should now be possible to add your command to dialogues through the Candy Dialogue Creator.

However, before recompiling Candy DC, check the following:

- Add the command to a Block and see if it displays the correct data fields, with the correct default values.
- Enter data in all fields (change the default values) and save/load the dialogue to check that it loads the correct data.

- Export a dialogue featuring the command, then open the exported .txt file and check that all the data is there for the command.
- Change to another Block, then return to the Block featuring the command and see if it displays the correct data.
- Add more commands to the Block, scroll up or down until your custom command disappears from view, then scroll back to it and check if it the data displays correctly. In the process, check also that other lines don't display incorrectly (some errors can cause a command's data to display in other commands upon scrolling).
- Cut/Copy/Paste the command to another position in the Block.
- In the Writer UI, check that the filters show/hide your command correctly, and that the command displays correctly in the list.

The most likely sources of errors would be:

- Custom data fields not being set to invisible by default in **_apply_line_data_to_ui()**.
- Custom data fields not being set to visible in the command's block, in **_apply_line_data_to_ui()** (remember to make the parent category of data fields visible too!).
- Custom data fields not updating values correctly in the command's block, in **_apply_line_data_to_ui()**.
- Errors in the input functions for custom data fields, such as updating the wrong data key when values are entered (check that the spelling of data keys matches with the command's keys in the **command_templates** dictionary).
- Data field signals not connected to the data fields input functions (or connected to the wrong functions, if you duplicated fields).
- Missing or excess '§' prefix in the command name somewhere.
- Errors in the **line_colors** dictionary (you may have forgotten to add your command there entirely).
- Any issues that only appear in the Writer UI are likely caused by errors in either the **line_colors** dictionary or the **Writer.gd** script. Errors in **Universal_Line.gd** would almost certainly cause errors in the Main UI as well.